

Relatório Técnico — Decibéis: a turnê contra O Empresário

Entregável DEL-05 — Relatório em Markdown. **Parte humana (D-02):** Exportar para PDF via VS Code ("Markdown PDF" ou "Print to PDF") ou Pandoc — o agente produz apenas o texto-fonte.

Disciplina: Programação Orientada a Objetos — UTFPR · 2026 **Autor:** João Prissão (JoaoPrissao)

Repositório: branch `modo-historia`

Sumário

1. Introdução
 2. Requisitos Funcionais do PDF
 3. Arquitetura
 4. Padrões de Projeto
 5. Herança e Polimorfismo
 6. Sobrecarga de Operadores
 7. Qualidade e Testes
 8. Decisões de Design
 9. Conclusão
-

1. Introdução

1.1 Objetivo do Projeto

Este projeto implementa um jogo de RPG de gerenciamento de banda de rock como trabalho final da disciplina de Programação Orientada a Objetos. O objetivo é demonstrar — em código executável — os principais conceitos da POO: herança, polimorfismo, encapsulamento, abstração, padrões de projeto GoF e sobrecarga de operadores.

O projeto foi desenvolvido em **Python 3.12** com interface web rodando via **pywebview**, mantendo o domínio de jogo em Python puro e o frontend em HTML/CSS/JS responsável apenas pela renderização.

1.2 Tema Escolhido: Banda de Rock em Turnê

Em vez do RPG fantasia clássico (guerreiros/magos), o tema é uma **banda de rock em turnê**: quatro músicos (Guitarrista, Vocalista, Baterista, Baixista) percorrem venues (bar, feira, arena) vencendo capangas e o chefe final O Empresário. O resultado é um jogo jogável de ponta a ponta:

- **Overworld 2D:** van da banda percorre um mapa side-scroll; o jogador para em venues, conversa com NPCs e abre baús.
- **Batalha por turnos:** cada músico ataca com 3 golpes (leve/médio/pesado); o vilão revida; sistema de energia e cansaço gere a estratégia.
- **Minigame de ritmo:** antes de cada ataque o jogador pressiona teclas no ritmo certo para multiplicar o dano — acertos perfeitos geram combos.

2. Requisitos Funcionais do PDF

O PDF da disciplina especifica dois requisitos funcionais centrais: **criação de personagem com escolhas** e **level-up com pontos de experiência**. Esta seção documenta como esses requisitos foram atendidos e/ou reinterpretados tematicamente no contexto do jogo de banda (DEL-07).

2.1 Criação de Personagem com Escolhas

Requisito original: O jogador cria um personagem escolhendo raça/classe com atributos iniciais distintos.

Reinterpretação temática: Em vez de um único protagonista, o jogador monta uma **banda de 4 músicos**, escolhendo nome e tipo para cada membro. Cada tipo tem atributos iniciais diferentes:

Tipo	Atributo-chave	Valor inicial	Mecânica de ataque
Guitarrista	força	10	$\text{força} * 1.5 + \text{ego}$
Vocalista	inteligencia	10	$\text{inteligencia} * 1.8$
Baterista	agilidade	10	$\text{agilidade} * 1.2 + \text{chance de crítico}$
Baixista	força (via Guitarrista) + fe	—	$\text{força} * 1.3 + \text{fe} * 0.5$

Evidência no código: - `backend/fabricas.py` — `MusicoFactory.criar(tipo, nome=...)` instancia a subclasse correta. - `bridge/api.py` — `criar_banda(composicao)` recebe a lista de `{tipo, nome}` e chama a factory. - O frontend exibe a tela `#tela-criar-banda` onde o jogador preenche os 4 membros.

Esta abordagem cobre o requisito de "criação de personagem com escolhas" de forma temática: a "escolha de personagem" é a composição da banda.

2.2 Level-up com Pontos de XP

Requisito original: Personagem acumula pontos de experiência (XP) e sobe de nível, ganhando atributos.

Reinterpretação temática: XP representa a **experiência de turnê** — shows realizados com sucesso. Subir de nível é a **reputação crescente do músico**; o bônus de HP (+10 por nível) representa a resistência física que um músico profissional desenvolve em shows ao vivo.

Evidência no código (`backend/musico.py`):

```
def ganhar_xp(self, quantidade: int) → None:
    if quantidade ≤ 0:
        return
    self._xp += quantidade
    while self._xp ≥ self._xp_proximo_nivel:
        self._xp -= self._xp_proximo_nivel
        self.subir_nivel()

def subir_nivel(self) → None:
    self._nivel += 1
    self._xp_proximo_nivel = self._nivel * self.XP_BASE # 100, 200, 300...
    bonus_hp = 10
    self._hp_maximo += bonus_hp
    self._hp = min(self._hp + bonus_hp, self._hp_maximo)
```

XP é concedido ao concluir uma venue (`concluir_venue()` em `bridge/api.py`). O level-up é automático ao atingir o limiar — sem ponto de gasto manual, pois a progressão é orgânica à turnê. O XP acumulado além do limiar é preservado e carregado para o próximo nível, mantendo coerência em subidas múltiplas de nível de uma vez.

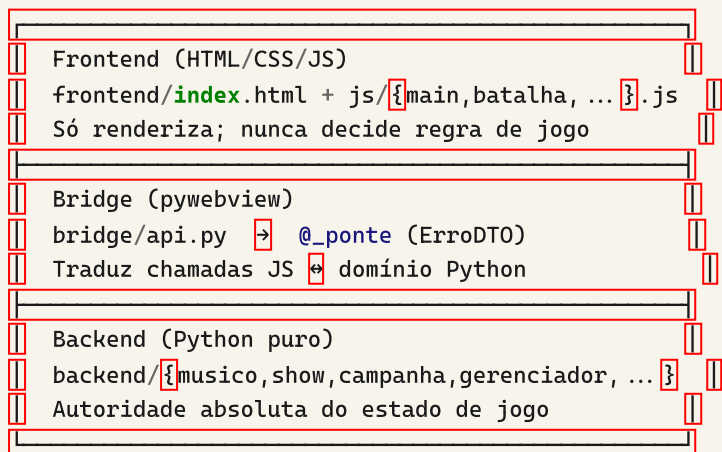
2.3 Resumo da Reinterpretação

Requisito do PDF	Implementação no jogo	Arquivo de evidência
Criação de personagem com escolhas	Composição da banda (4 músicos, tipos distintos)	<code>fabricas.py</code> , <code>api.py</code>
Level-up com XP	Experiência de turnê → reputação do músico → +HP	<code>musico.py</code> (<code>ganhar_xp</code> , <code>subir_nivel</code>)

3. Arquitetura

3.1 Visão Geral — Três Camadas

O projeto segue uma arquitetura em **três camadas** com separação estrita de responsabilidades:



Backend — lógica de domínio em Python OOP puro. Sem I/O de UI; comunica erros via exceções. **Bridge** — `@_ponte` captura qualquer exceção do domínio e converte para DTO JSON `{ok, erro}`. **Frontend** — renderiza estado recebido do backend; não valida regras, não mantém estado canônico.

3.2 Diagrama UML de Classes

O diagrama completo está em `docs/uml_classes.md` — bloco Mermaid `classDiagram` coerente com a implementação real, incluindo as três hierarquias (Musico, Item, JogoError) e as relações de Singleton, Factory e Bridge.

4. Padrões de Projeto

Foram identificados e documentados **6 padrões GoF** no código. Os 4 primeiros são os canônicos exigidos pela rubrica (DEL-06); os 2 últimos são bônus.

4.1 Singleton — GerenciadorJogo

Categoria GoF: Criacional **Arquivo:** `backend/gerenciador.py`

Garante que existe **uma única instância** do estado central do jogo em toda a execução.

```
class GerenciadorJogo:
    _instancia: "GerenciadorJogo | None" = None

    def __new__(cls) → "GerenciadorJogo":
        if cls._instancia is None:
            cls._instancia = super().__new__(cls)
            cls._instancia._inicializado = False
        return cls._instancia
```

Qualquer módulo que instancie `GerenciadorJogo()` recebe o mesmo objeto. Testado em `tests/test_singleton.py`.

4.2 Factory Method — MusicoFactory e ItemFactory

Categoria GoF: Criacional **Arquivo:** `backend/fabricas.py`

Centraliza a criação de músicos e itens a partir de uma string de tipo. O dicionário `_tipos` (ou `_catalogo`) é o **único ponto que cita classes concretas** — novos tipos são adicionados via `registrar()` sem alterar o código existente (princípio Open/Closed).

```

class MusicoFactory:
    _tipos: dict[str, type[Musico]] = {
        "guitarrista": Guitarrista,
        "vocalista": Vocalista,
        "baterista": Baterista,
        "baixista": Baixista,
    }

    @classmethod
    def criar(cls, tipo: str, **kwargs) → Musico:
        if tipo not in cls._tipos:
            raise TipoInvalidoError(tipo)
        return cls._tipos[tipo](**kwargs)

    @classmethod
    def registrar(cls, chave: str, classe: type[Musico]) → None:
        cls._tipos[chave] = classe

```

ItemFactory segue a mesma estrutura para itens (Equipavel, Consumivel). Testado em tests/test_fabricas.py.

4.3 Template Method — Musico (ABC) e Subclasses

Categoria GoF: Comportamental **Arquivo:** backend/musico.py + Guitarrista.py, Vocalista.py, Baterista.py, Baixista.py

Musico é uma classe abstrata (ABC) que define o **esqueleto do algoritmo** de personagem: HP, XP, energia, level-up, equipamento — todos implementados na classe base. O único **passo variável** é atacar(), obrigatoriamente sobrescrito por cada subclasse.

```

class Musico(ABC):
    @abstractmethod
    def atacar(self) → int:
        """Retorna o dano base sem aplicá-lo."""
        pass

    def subir_nivel(self) → None: # algoritmo fixo herdado por todos
        self._nivel += 1
        self._xp_proximo_nivel = self._nivel * self.XP_BASE
        ...

```

Cada subclasse implementa atacar() de forma específica ao seu tipo (força, inteligência, agilidade) — isso é o Template Method: estrutura fixa na base, passo concreto nas folhas.

4.4 Strategy — Moveset (moves.py)

Categoria GoF: Comportamental **Arquivo:** backend/moves.py + backend/show.py

O jogador escolhe **qual golpe usar em cada turno** (leve, médio, pesado, especial de equipamento). Cada golpe é um dicionário que encapsula a estratégia de ataque — multiplicador de dano, custo de energia, efeito de cansaço:

```
MOVES_BASE = {
    "guitarrista": [
        {"id": "leve", "mult": 0.8, "custo": 10, "cansa": False},
        {"id": "medio", "mult": 1.0, "custo": 25, "cansa": False},
        {"id": "pesado", "mult": 1.5, "custo": 40, "cansa": True},
    ],
    ...
}
```

`Show.acao_musico()` recebe o move e aplica `mult_extra`, `custo_energia` e `cansa` sem saber qual move foi escolhido — é o padrão Strategy: o algoritmo de ataque é selecionado em runtime.

4.5 Hierarquia de Exceções — `JogoError` (Composite de tipos)

Categoria GoF: Estrutural (hierarquia Composite) **Arquivo:** `backend/excecoes.py`

Toda exceção do jogo deriva de `JogoError`, formando uma **árvore tipada de 26 classes** (1 raiz + 25 subclasses). Isso permite capturar erros no nível certo: `except JogoError` para tudo, `except InventarioError` só para erros de inventário.

```
class JogoError(Exception): pass # raiz
class InventarioError(JogoError): pass # ramo
class InventarioCheioError(InventarioError): pass # folha
class ItemNaoEncontradoError(InventarioError): pass
class PersistenciaError(JogoError): pass
class SaveNaoEncontradoError(PersistenciaError): pass
class CampanhaError(JogoError): pass
class VenueBloqueadaError(CampanhaError): pass
# ... 26 classes no total (1 raiz + 25 subclasses)
```

Testado em `tests/test_excecoes.py`.

4.6 Bridge — Ponte JS ↔ Domínio Python (bônus)

Categoria GoF: Estrutural **Arquivo:** `bridge/api.py`

O decorator `@_ponte` separa a **abstração** (API JS-friendly — métodos públicos com retorno JSON) da **implementação** (domínio Python puro com exceções). O frontend chama `window.pywebview.api.atacar( ... )` sem conhecer nada do domínio:

```
def _ponte(fn):
    def wrapper(*args, **kwargs):
        try:
            return {"ok": True, "dados": fn(*args, **kwargs)}
        except JogoError as e:
            return {"ok": False, "erro": str(e)}
    return wrapper
```

Nenhuma exceção cruza a ponte — o frontend recebe sempre `{ok, dados}` ou `{ok: false, erro}`.

Tabela Resumo dos Padrões

#	Padrão	Categoria GoF	Arquivo	Símbolo de Evidência
1	Singleton	Criacional	backend/gerenciador.py	<code>_instancia</code> + <code>__new__</code>
2	Factory Method	Criacional	backend/fabricas.py	<code>MusicoFactory.criar()</code> + <code>registrar()</code>
3	Template Method	Comportamental	backend/musico.py + subclasses	<code>ABC</code> + <code>@abstractmethod atacar()</code>
4	Strategy	Comportamental	backend/moves.py + show.py	<code>MOVES_BASE</code> + move escolhido em runtime
5	Hierarquia de Exceções	Estrutural	backend/excecoes.py	<code>JogoError</code> raiz de 26 classes (1 raiz + 25 subclasses)
6	Bridge	Estrutural	bridge/api.py	<code>@_ponte</code> — separa API JS do domínio Python

5. Herança e Polimorfismo

5.1 Hierarquia Musico → [Guitarrista, Vocalista, Baterista, Baixista]

```
Musico (ABC)
├── Guitarrista
│   └── Baixista ← herda de Guitarrista (compartilha atributo forca)
├── Vocalista
└── Baterista
```

Herança de implementação: todos herdam de `Musico` o mecanismo completo de HP, XP, energia, level-up, inventário e equipamento. Cada subclasse acrescenta apenas seu atributo específico.

Observação arquitetural: `Baixista` herda de `Guitarrista` (não direto de `Musico`) porque o baixista compartilha o atributo `forca` da guitarra, acrescentando `fe` como recurso diferenciador. Isso evita duplicação de código.

5.2 Hierarquia Item → [Equipavel, Consumivel]

```
Item (base)
├── Equipavel ← ocupa slot, bônus de atributo, não é destruído ao usar
└── Consumivel ← efeito único (cura/energia), destruído ao usar
```

`Equipavel` sobrescreve `usar(alvo)` para aplicar o bônus de atributo e validar `classes_permitidas`. `Consumivel` sobrescreve `usar(alvo)` para aplicar `cura` ou `energia` e marcar `consumir_ao_usar = True`.

5.3 Polimorfismo em Ação

atacar() **polimorfo:** `Show.acao_musico(musico, move)` chama `musico.atacar()` sem saber o tipo concreto — Guitarista, Vocalista, Baterista ou Baixista. O resultado é o dano base específico daquele tipo, multiplicado pelo fator do move escolhido.

usar(alvo) **polimorfo:** `Inventario.usar(item_id, alvo)` chama `item.usar(alvo)` sem saber se é `Equipavel` ou `Consumivel` — cada um executa seu efeito específico.

6. Sobrecarga de Operadores

6.1 `Inventario.__len__`

Classe: `Inventario` (`backend/inventario.py`) **Dunder:** `__len__` **Semântica:** `len(inv)` retorna o número de itens no inventário.

```
def __len__(self) → int:
    return len(self._itens)
```

Pré-existente; usado na verificação de capacidade e nos testes.

6.2 `Inventario.__contains__` (adicionado em 04-01)

Classe: `Inventario` (`backend/inventario.py`) **Dunder:** `__contains__` **Semântica:** `"Pedal de Efeito" in inventario` — busca por nome, delega para `_buscar()`.

```
def __contains__(self, nome_item: str) → bool:
    """Permite o idioma Pythônico: 'Pedal de Efeito' in inventario."""
    return self._buscar(nome_item) is not None
```

Adicionado via TDD em 04-01: testes em `tests/test_inventario.py` (`test_contains_*`).

6.3 `Inventario.__repr__` (adicionado em 04-01)

Classe: `Inventario` (`backend/inventario.py`) **Dunder:** `__repr__` **Semântica:** `repr(inv)` → `"Inventario(3/20 itens)"` — representação legível para debug.

```
def __repr__(self) → str:
    return f"Inventario({len(self._itens)}/{self.capacidade} itens)"
```

Adicionado via TDD em 04-01: testes em `tests/test_inventario.py` (`test_repr_*`).

6.4 `Musico.__del__`

Classe: `Musico` (`backend/musico.py`) **Dunder:** `__del__` **Semântica:** destrutor — print de ciclo de vida ao remover o objeto da memória.


```
def __del__(self):
    print(f" [-] Músico '{self._nome}' removido da memória.")
```

Demonstra gerenciamento de ciclo de vida do objeto; útil para debug de memory leaks.

Tabela Resumo dos Dunders

Dunder	Classe	Semântica
<code>__len__</code>	Inventario	<code>len(inv)</code> — contagem de itens
<code>__contains__</code>	Inventario	<code>"nome" in inv</code> — busca por nome
<code>__repr__</code>	Inventario	<code>repr(inv)</code> → <code>"Inventario(N/cap itens)"</code>
<code>__del__</code>	Musico	destrutor — ciclo de vida

7. Qualidade e Testes

7.1 Suite pytest

Métrica	Valor
Testes passando	361
Testes falhando	0
Cobertura TOTAL (backend/)	95%
backend/main.py	omitido via <code>.coveragerc</code> (script CLI demo, 0% cobertura)
Limiar da rubrica (DEL-02)	≥ 60% — muito acima

Cobertura original (com `main.py`): 86%. Com `.coveragerc` omitindo `main.py` (script de demo CLI, nunca executado pelos testes de domínio): 95%. Ambos muito acima do limiar de 60%.

Comando para reproduzir:

```
pytest --cov=backend --cov-report=term-missing -q
```

Os 361 testes cobrem: musicos (criação, atributos, level-up, energia), inventário (add/remove/use, dunder), itens (equipavel/consumivel), fábricas (MusicoFactory/ItemFactory), combate (Show, golpes, cansaço, crítico), campanha (venues, fama, cache, cooldowns), persistência (round-trip JSON), exceções, API bridge, integração.

7.2 Falhas Pré-existentes de Harness (documentação honesta)

Existem **2 falhas conhecidas nos harnesses JS** que não foram corrigidas nesta entrega — estão fora do escopo por decisão de projeto (D-06).

Falha 1 — `batalha.harness.html` : **derrota chama `parar()`** - Comportamento esperado: ao perder o show, o harness chama `parar()` para interromper o clock de auto-ataque. - Comportamento atual: `parar()` não é chamado corretamente / timing divergente. - **Impacto:** Apenas no harness de teste. O jogo real funciona corretamente — a batalha termina como esperado ao jogar pelo `python bridge/app.py`. - **Decisão:** Corrigir está fora do escopo desta entrega.

Falha 2 — `ritmo.harness.html` : **`void 0`** - Comportamento esperado: função de callback retorna um valor. - Comportamento atual: retorna `undefined` (`void 0`) em certas condições do harness. - **Impacto:** Apenas no harness de teste. O minigame de ritmo funciona corretamente no app real. - **Decisão:** Corrigir está fora do escopo desta entrega.

Estas falhas **não afetam a jogabilidade real** nem a cobertura dos testes pytest (que testam o backend Python, não os harnesses JS).

8. Decisões de Design

As decisões abaixo foram tomadas ao longo do desenvolvimento e documentadas no STATE.md do projeto.

Decisão	Racional	Status
Backend autoritativo; frontend só renderiza	Separa claramente domínio de apresentação; torna o domínio testável sem UI	Confirmado
<code>Baixista</code> herda de <code>Guitarrista</code>	Compartilha <code>forca</code> ; evita duplicação de atributo e métodos <code>aumentar_forca</code> / <code>bonus Equipamento</code>	Confirmado
<code>@_ponte</code> captura toda exceção do domínio	Nenhuma exceção crua cruza para o JS; frontend sempre recebe <code>{ok, dados}</code> ou <code>{ok: false, erro}</code>	Confirmado
Energia unificada na classe base (<code>Musico</code>)	Simplifica o modelo — todos os tipos usam o mesmo recurso de energia, sem estados paralelos	Confirmado
Slots de equipamento reversíveis	<code>desequipar()</code> remove o item do slot; bônus é calculado dinamicamente em <code>atacar()</code> via <code>bonus_equipamento()</code>	Confirmado
<code>ItemFactory</code> com lambdas como fábricas	Lambdas com <code>**kw</code> permitem customização de atributos (ex: baú dá item com bônus maior) sem subclasses extras	Confirmado
Loja como ponto do mapa (não na van)	Feedback do João no UAT: van só equipa/usa/guarda itens já adquiridos; compra acontece em ponto fixo	Confirmado
<code>MusicoCantorError</code> não necessário	Fôlego do Vocalista foi unificado em <code>_energia</code> ; alias é derivado puro, sem estado extra	Confirmado

9. Conclusão

O projeto **Decibéis** demonstra com sucesso os conceitos centrais de POO em Python 3.12:

- **Herança** em duas hierarquias reais (`Musico` → 4 subclasses ; `Item` → 2 subtipos), com uma relação não-óbvia (`Baixista` herda de `Guitarrista`).
- **Polimorfismo** em `atacar()` e `usar()` — o código cliente nunca precisa saber o tipo concreto.
- **Encapsulamento** — atributos protegidos (`_`) e privados (`__`) com acesso via getters/setters com validação.
- **Abstração** — `Musico` como ABC define a interface; `Item` como base define o contrato.
- **4+ Padrões GoF** documentados com evidências em código: Singleton, Factory Method, Template Method, Strategy, Hierarquia de Exceções, Bridge.
- **Sobrecarga de operadores** (`__len__` , `__contains__` , `__repr__` , `__del__`) tornando as classes mais Pythônicas.
- **Cobertura de testes** de 95% (`backend/`) com 361 testes pytest verdes.

O jogo é jogável de ponta a ponta: menu → criar banda → overworld 2D → batalha com minigame de ritmo → level-up → loja → progressão até O Empresário chefe final.

Referências

- Gamma, E. et al. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
- Python Software Foundation. *Python 3.12 Data Model — Special method names*. docs.python.org.
- pywebview. *Documentation 6.x*. pywebview.flowrl.com.
- Especificação da Disciplina POO — UTFPR 2026 (ver `docs/P00___Especificação_do_Projeto_da_Disciplina.pdf`).